
Как устроена агентная система

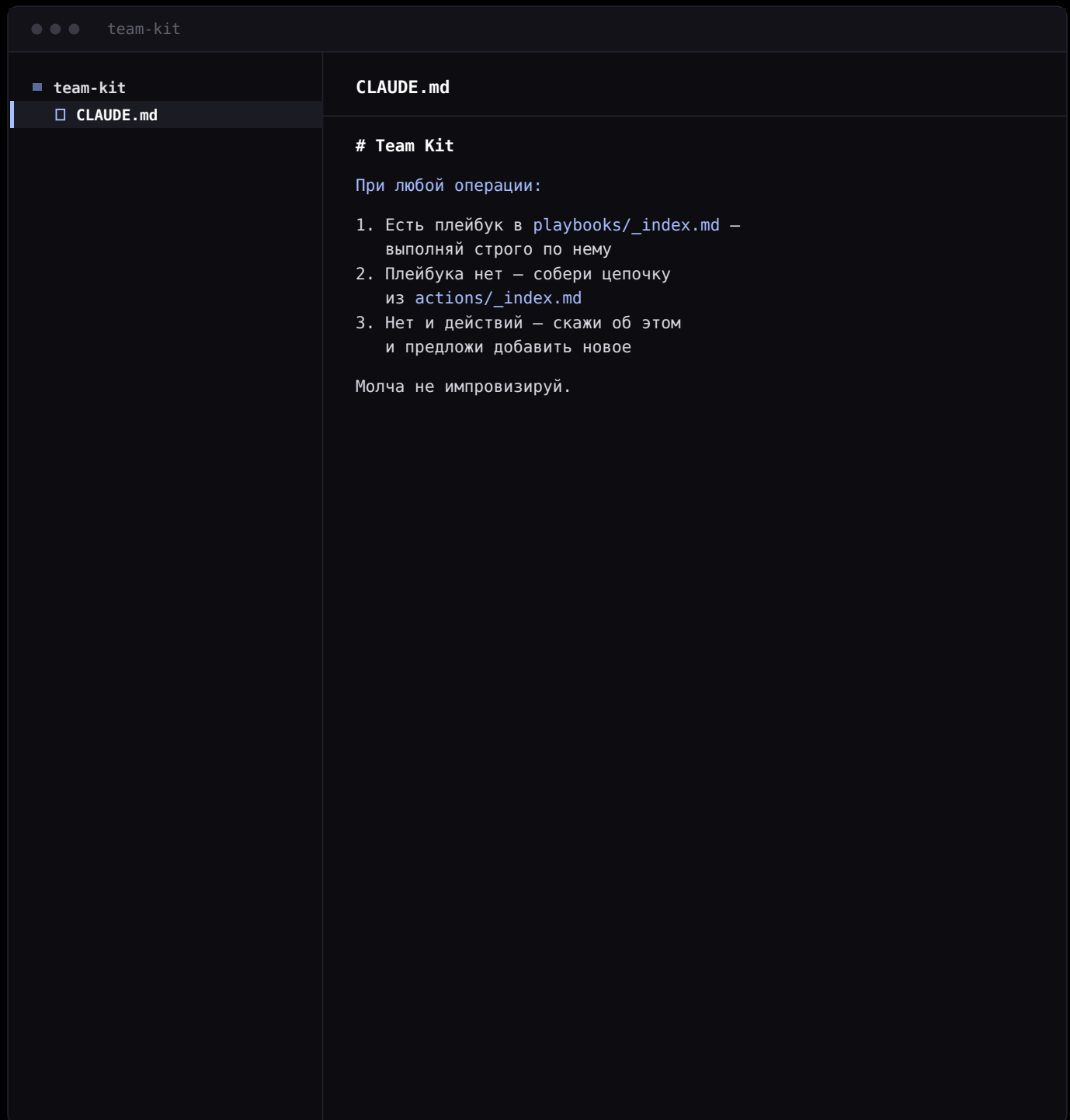
Восемь файлов на примере команды разработки.
По одному файлу на каждый слой архитектуры.

Материалы вебинара

Как LLM меняет процесс разработки



Точка входа



```
team-kit

team-kit
  CLAUDE.md

# Team Kit

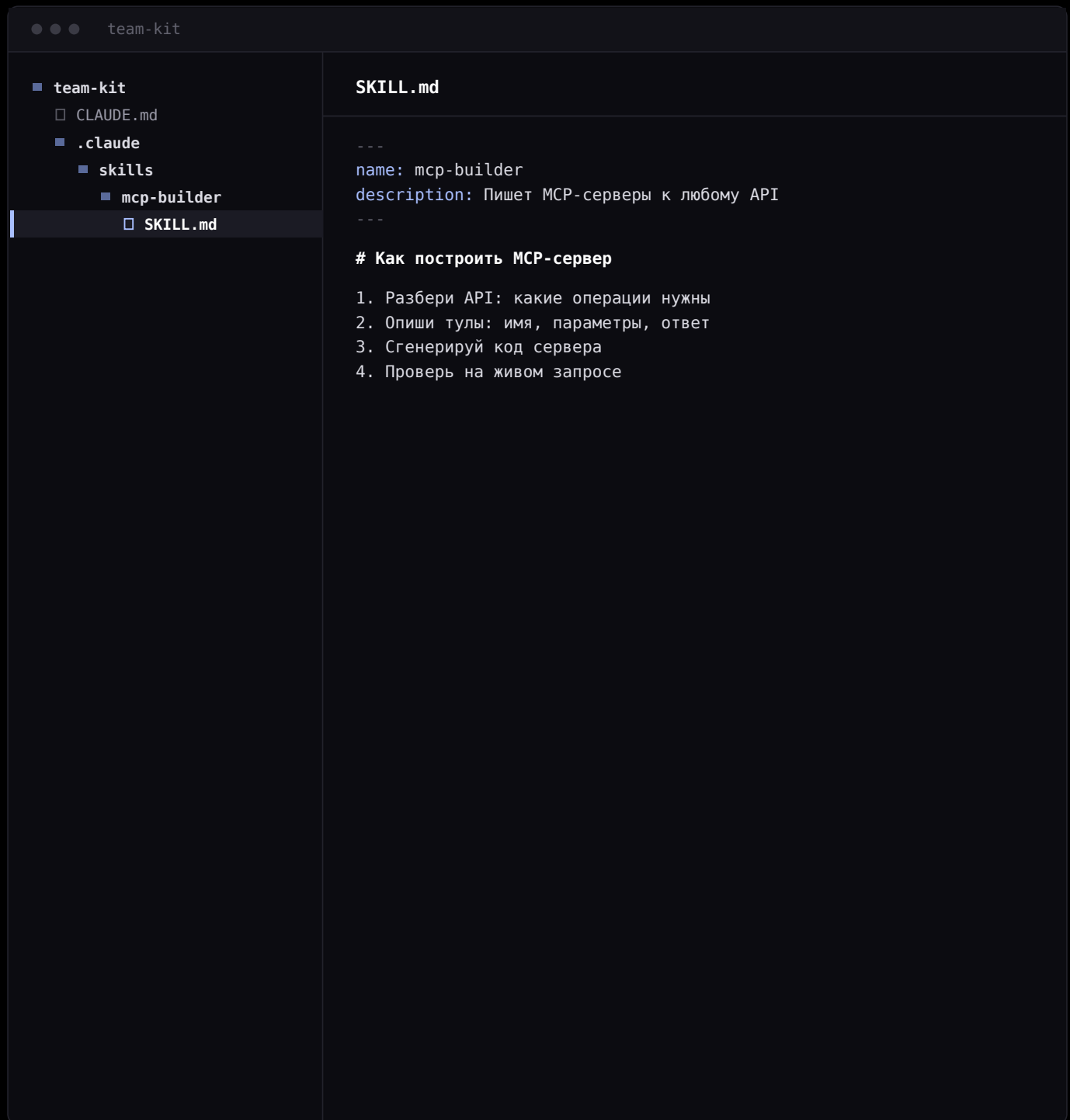
При любой операции:

1. Есть плейбук в playbooks/_index.md –
   выполняй строго по нему
2. Плейбука нет – собери цепочку
   из actions/_index.md
3. Нет и действий – скажи об этом
   и предложи добавить новое

Молча не импровизируй.
```

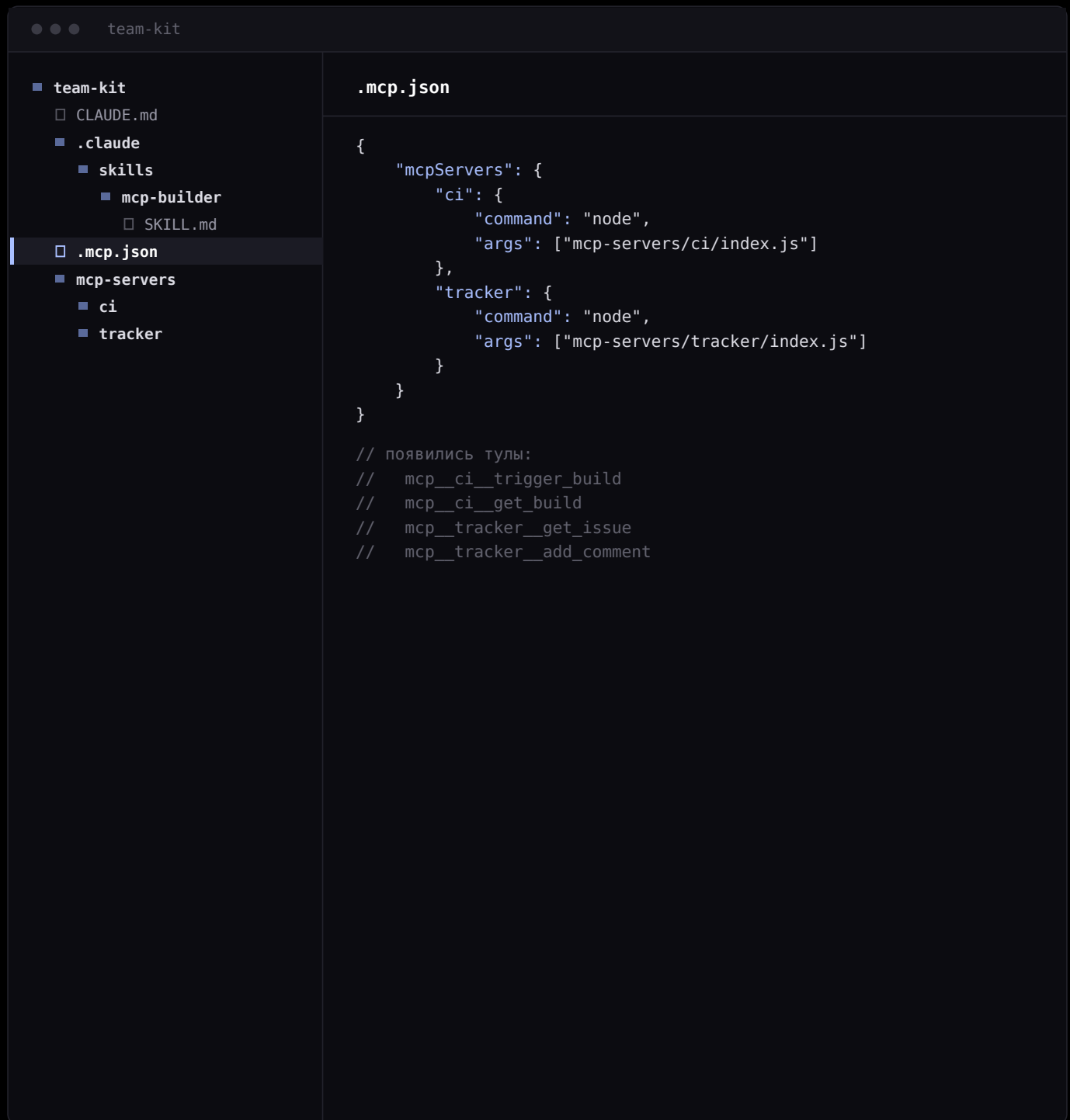
Единственный файл, который агент читает сам, без просьбы. Три ступени: готовый сценарий → сборка из действий → честное «нечем, давай добавим».

Скилл, который пишет серверы



Скилл — папка с инструкцией, которую агент подхватывает сам. Готовый берет из открытых наборов и просят словами: «сделай сервер к нашему CI — запустить сборку, статус, лог».

Руки



```
team-kit

team-kit
├── CLAUDE.md
├── .claude
│   └── skills
│       └── mcp-builder
│           └── SKILL.md
├── .mcp.json
├── mcp-servers
│   ├── ci
│   └── tracker
```

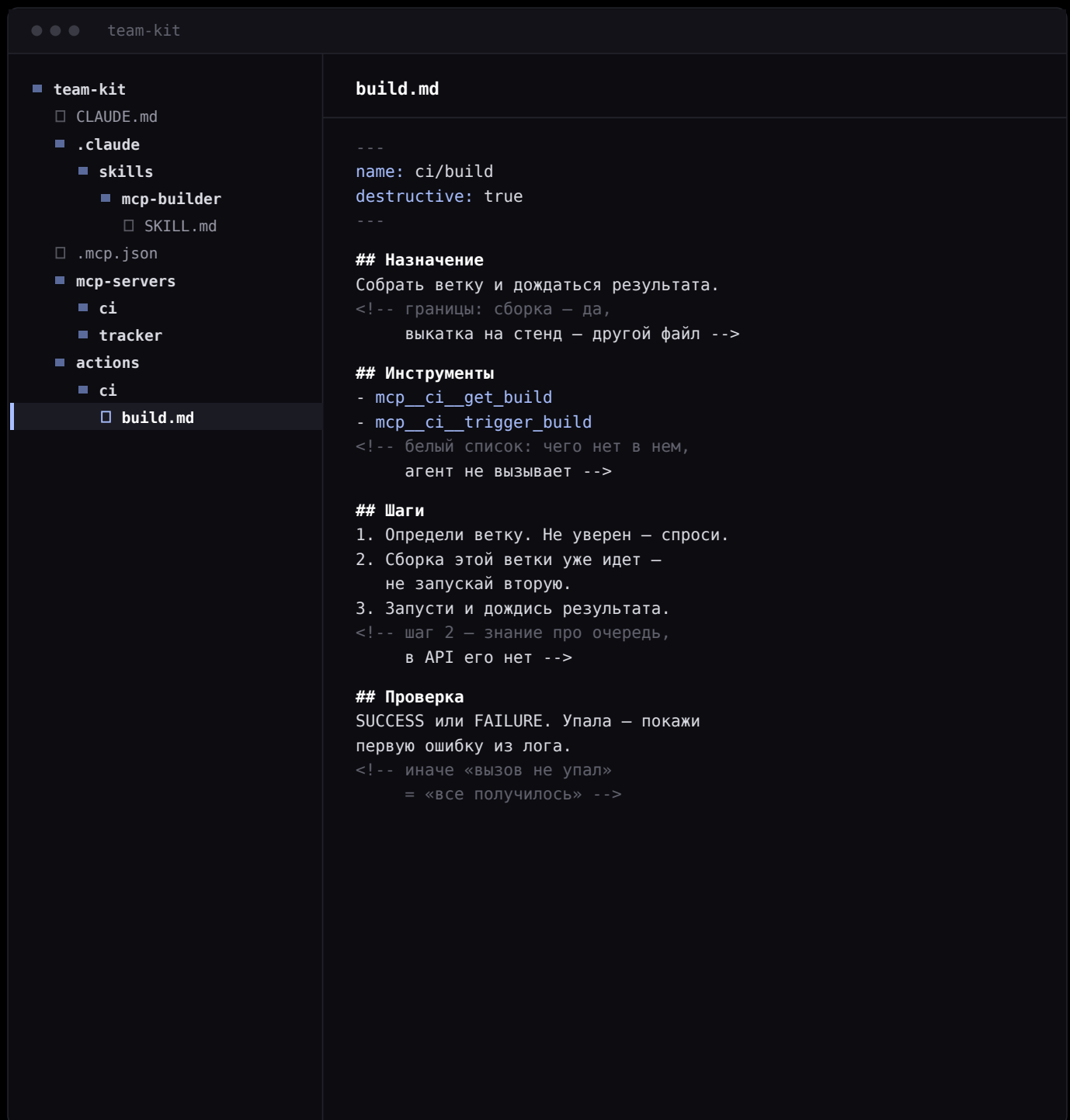
```
.mcp.json

{
  "mcpServers": {
    "ci": {
      "command": "node",
      "args": ["mcp-servers/ci/index.js"]
    },
    "tracker": {
      "command": "node",
      "args": ["mcp-servers/tracker/index.js"]
    }
  }
}

// появились тулы:
// mcp_ci_trigger_build
// mcp_ci_get_build
// mcp_tracker_get_issue
// mcp_tracker_add_comment
```

Теперь агент умеет запускать сборку и писать в задачу. Но не знает, когда сборку запускать не надо и что писать в комментарии.

Первое действие



```
team-kit
├── CLAUDE.md
├── .claude
│   └── skills
│       └── mcp-builder
│           └── SKILL.md
├── .mcp.json
├── mcp-servers
│   ├── ci
│   └── tracker
├── actions
│   └── ci
└── build.md
```

```
build.md

---
name: ci/build
destructive: true
---

## Назначение
Собрать ветку и дождаться результата.
<!-- границы: сборка — да,
      выкатка на стенд — другой файл -->

## Инструменты
- mcp__ci__get_build
- mcp__ci__trigger_build
<!-- белый список: чего нет в нем,
      агент не вызывает -->

## Шаги
1. Определи ветку. Не уверен — спроси.
2. Сборка этой ветки уже идет —
   не запускай вторую.
3. Запусти и дождись результата.
<!-- шаг 2 — знание про очередь,
      в API его нет -->

## Проверка
SUCCESS или FAILURE. Упала — покажи
первую ошибку из лога.
<!-- иначе «вызов не упал»
      = «все получилось» -->
```

Тул — что можно сделать. Действие — как это делать правильно. Комментарии в файле — не украшение: они объясняют следующему, зачем шаг, и его не выкидывают при правке.

Второе действие

team-kit

- team-kit
 - CLAUDE.md
 - .claude
 - skills
 - mcp-builder
 - SKILL.md
 - .mcp.json
 - mcp-servers
 - ci
 - tracker
 - actions
 - ci
 - build.md
 - tracker
 - comment.md

comment.md

```
---
name: tracker/comment
destructive: true
---

## Назначение
Опубликовать комментарий в задаче.
<!-- границы: комментарий — да,
      смена статуса — другой файл -->

## Инструменты
- mcp__tracker__add_comment
<!-- белый список: чего нет в нем,
      агент не вызывает -->

## Шаги
1. Суть первой строкой, факты, ссылки.
2. Покажи превью и дождись
   подтверждения — коммент видят все.
3. Вызови add_comment(issue, text).
<!-- шаг 2 — мягкий стоп-кран -->

## Проверка
Вернулся id — опубликован.
Нет — покажи ошибку, не повторяй.
<!-- иначе «вызов не упал»
      = «все получилось» -->
```

Каркас тот же: назначение, инструменты, шаги, проверка. Одно действие — один результат. Комментарий и смена статуса живут в разных файлах.

Реестры

team-kit

- team-kit
 - CLAUDE.md
 - .claude
 - skills
 - mcp-builder
 - SKILL.md
 - .mcp.json
 - mcp-servers
 - ci
 - tracker
 - actions
 - _index.md**
 - ci
 - build.md
 - tracker
 - comment.md
 - playbooks
 - _index.md новый

actions/_index.md

```
<!-- собран из шапок файлов -->

# Действия

## ci
- ci/build – собрать ветку
- ci/status – статус сборки

## tracker
- tracker/comment – написать в задачу
- tracker/close – закрыть с резолюцией

## repo
- repo/create-mr – MR по шаблону
```

Свой реестр у действий, свой — у плейбуков. Оба собираются из шапок файлов. Агент читает строку на действие и открывает только нужный файл — весь текст в память не влезет.

Цепочка

team-kit

- CLAUDE.md
- .claude
 - skills
 - mcp-builder
 - SKILL.md
- .mcp.json
- mcp-servers
 - ci
 - tracker
- actions
 - _index.md
 - ci
 - build.md
 - tracker
 - comment.md
- playbooks
 - _index.md
- build-and-report.md

build-and-report.md

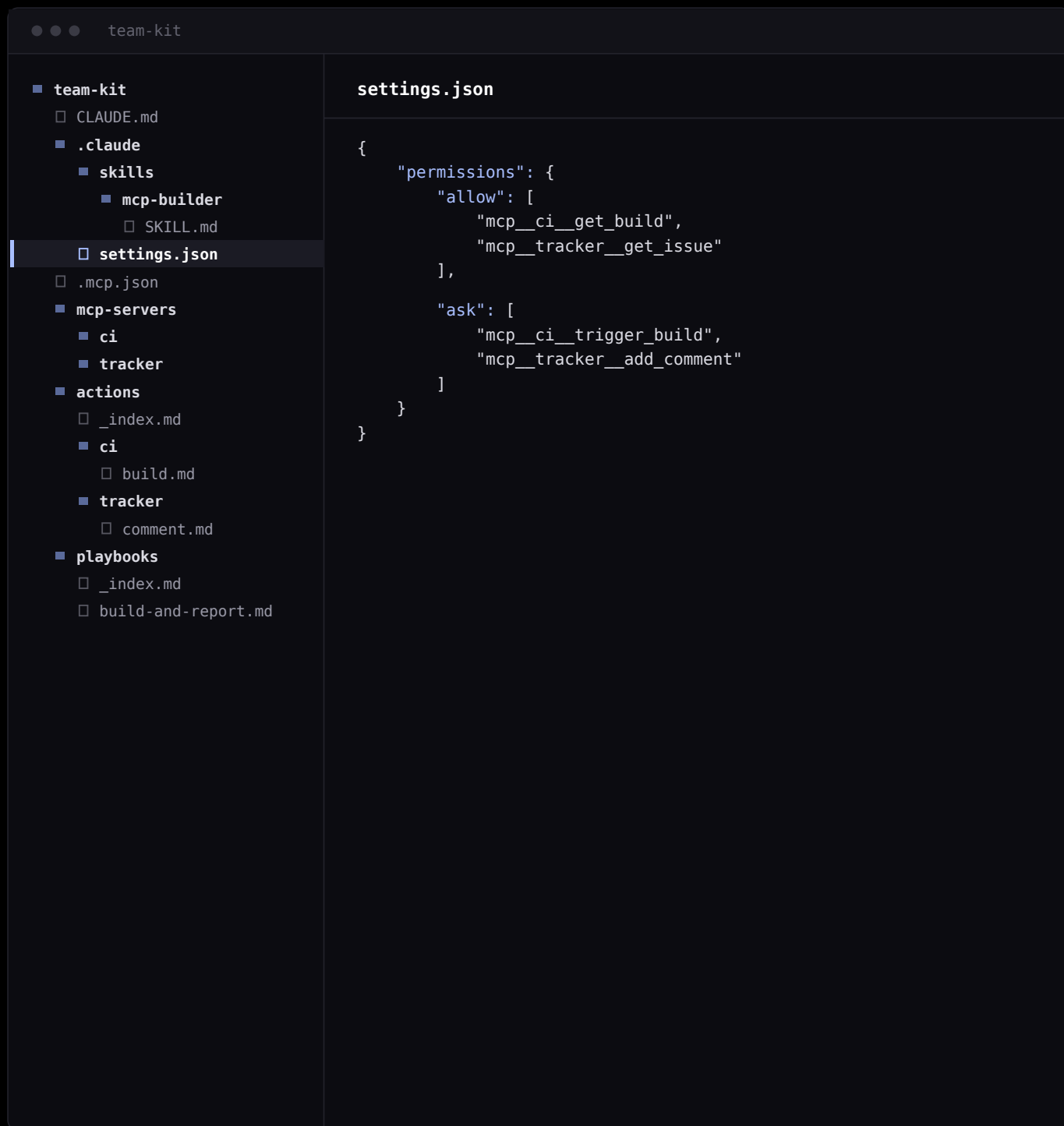
```
---
name: playbook/build-and-report
---

## Назначение
Собрать ветку и отписаться в задачу.

## Шаги
1. → actions/tracker/read-task.md
2. → actions/ci/build.md
3. → actions/tracker/comment.md
   шаблон «сборка успешна»
4. Покажи итог: номер сборки,
   ссылку, статус задачи.
<!-- плейбук не вызывает тулы,
      только ссылается на действия -->
```

Правка в `build.md` чинит все цепочки, где сборка участвует. Плейбука нет — агент соберет такую же цепочку сам из реестра действий.

Границы



Мягкая граница — слово «подтверди» внутри действия. Держится на том, что агент читает инструкцию. **Жесткая** — этот файл: посмотреть статус можно молча, запустить сборку — только с разрешения.

Что происходит по фразе

«собери ZB-451 и отпишись в задачу»

↓

CLAUDE.md

правило: сначала плейбуки

↓

playbooks/_index.md

есть build-and-report

↓

playbooks/build-and-report.md

4 шага, каждый — свое действие

↓

actions/ci/build.md

сначала проверь очередь, потом запускай

↓

.mcp.json

догружает серверы ci и tracker

↓

.claude/settings.json

trigger_build в списке «спросить»

↓

mcp_ci_trigger_build

вызов после ответа человека

Четыре слоя

слой	ГДЕ ЛЕЖИТ	ОТВЕЧАЕТ ЗА
Руки	mcp-servers/	что физически можно сделать
Знание	actions/	как это делать правильно
Маршрут	playbooks/, _index.md	что и в каком порядке
Границы	settings.json	где обязателен человек

Дальше добавляют по одному файлу

ФАЙЛ	ЧТО ЗАКРЫВАЕТ
actions/repo/create-mr.md	MR по шаблону, ревьюеры из состава команды
actions/tracker/close.md	закрытие задачи с резолюцией
playbooks/finish-task.md	MR, апрувы, сборка, статус — одной фразой
playbooks/duty.md	дежурство: сбор ошибок, отсев, раскладка по людям

Архитектура не меняется. Каждая пойманная ошибка становится строкой в файле действия — система растёт правкой текста, а не кодом.

Что дальше

СООБЩЕСТВО

Вступайте в Just AI Dev Community

Сообщество для разработчиков: практика, разборы и анонсы эфиров



SAI LA

Доступ к 350+ LLM

Один эндпоинт для всех моделей — тот самый, на котором работает наш слой

